

SYSTEM ARCHITECTURE AND DESIGN SPECIFICATION

1. Architectural Pattern: Model-View-Controller (MVC)

To ensure our application is maintainable and scalable, we have adopted the MVC pattern. This separation of concerns allows our team members to work on different layers of the application simultaneously.

- **Model (The Data):** This layer will handle the logic for book data and interaction with **LocalStorage**. It will define the "Textbook" object and manage saving/loading lists.
- **View (The Interface):** This layer consists of the HTML and CSS files developed from our UI/UX designs. It is responsible for rendering the search results and the "Post Listing" forms.
- **Controller (The Logic):** This JavaScript layer will act as the bridge. It will listen for user clicks (like "Search" or "Post"), tell the Model what data to fetch, and tell the View what to display.

2. Domain-Driven Design (DDD) Approach

We are using DDD to align our technical implementation with the real-world "Domain" of campus book trading. We have identified the following **Bounded Contexts**:

- **Listing Domain:** Focused on the "Seller" experience. This domain handles the creation and management of textbook entries, including condition and pricing logic.
- **Discovery Domain:** Focused on the "Buyer" experience. This domain handles search queries, course code filtering, and the retrieval of seller contact information.
- **Handover Domain:** Focused on the physical meeting at the **University of Regina campus**. This handles the logic for connecting users via email/phone and coordinating in-person swaps.

3. UML Data Schema (Domain Entities)

Before coding, we have modeled our primary domain entity to ensure data consistency across the team.

Class: Textbook Listing

- **listingID:** String (Unique identifier for management).
- **title:** String (Textbook name).
- **courseCode:** String (e.g., ENSE 281).
- **price:** Number (Targeting 30-70% savings).
- **condition:** String (New, Good, Fair).
- **sellerContact:** String (Email or Phone for connection).
- **status:** Enum (Available, Sold).

4. Process Flow (Sequence Diagram)

This diagram outlines how the system will function when a student uses the "Simple 4-Step Process":

1. **User** interacts with the **View** (clicks "Search").
2. **Controller** captures the input and queries the **Model**.
3. **Model** retrieves relevant items from **LocalStorage**.
4. **Controller** sends the data back to the **View** to be rendered for the user.